Auditable agentic data science pipeline

Project specification and build plan: architecture, pipeline stages, quality gates, evaluation, timeline, and how to build it with Claude Code

What it does. A user provides data plus a short context brief. The system ingests and profiles the data, cleans it, validates the result, plans an analysis grounded in the data and the user's intent, gets the plan approved, then runs exploratory and predictive analysis. Every step produces a report with the exact code, parameters, before/after statistics and check results, so the final output is both the analysis and a complete audit trail.

Positioning. Generic in architecture, benchmarked and pitched on credit/risk-style tabular data: an auditable automated analysis pipeline for regulated data.

Item	Decision
Agents	3 LLM agents: cleaning, review & planning, analysis
Deterministic stages	Ingestion & profiling; all execution happens through logged tools
Human involvement	Context brief at start; plan approval before analysis
Data scope (v1)	Tabular only: CSV, Excel, Parquet, JSON
Orchestration	LangGraph (typed state, checkpoints, human-in-the-loop interrupts)
Timeline	About 12 weeks at 15–20 h/week, including buffer
Publishing	Curated static dashboards on GitHub Pages behind a publish gate

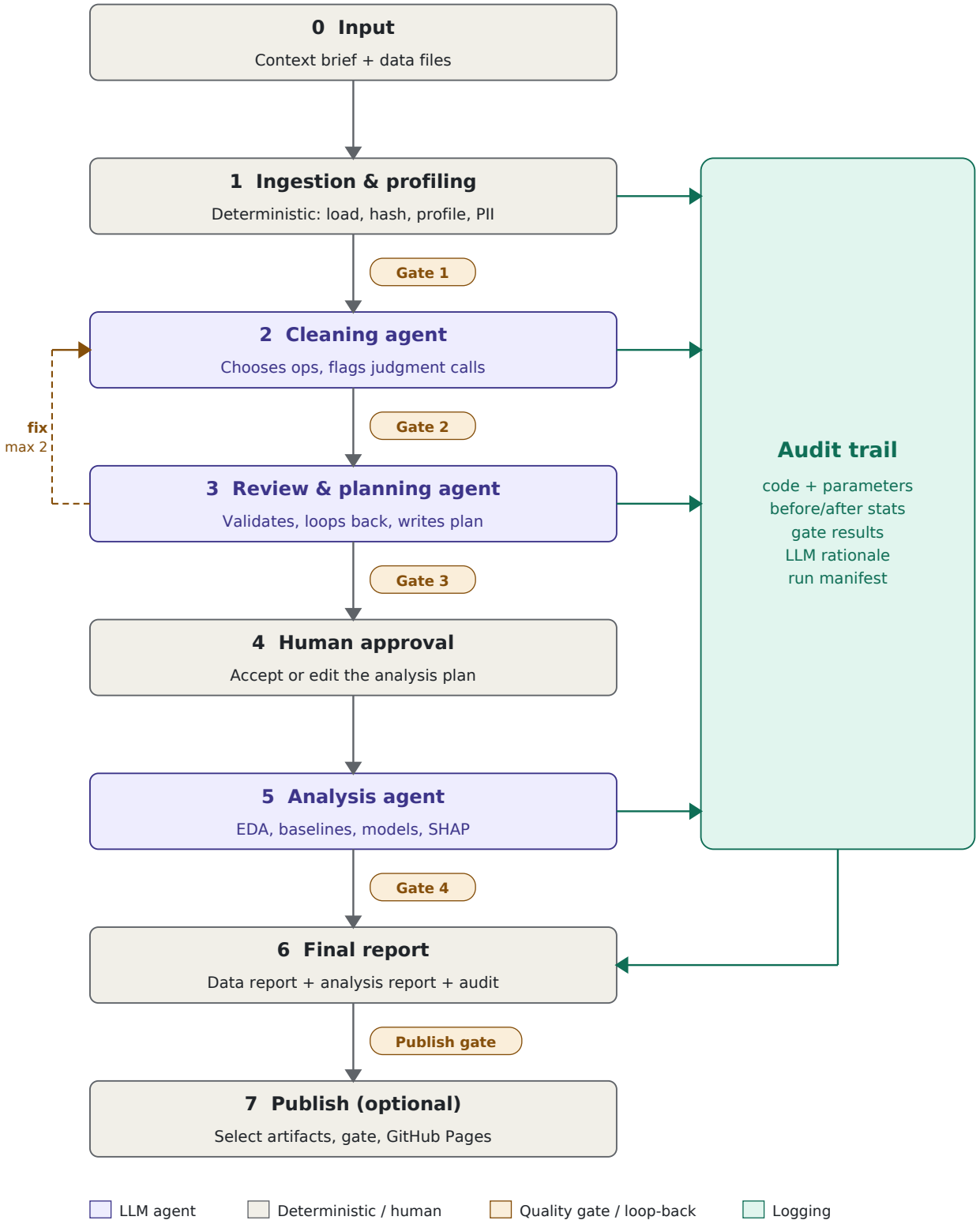
1. Design principles

These principles drive every later decision. When in doubt during the build, come back to them.

- **The LLM decides, tools execute.** Agents choose operations and write rationale; tested Python functions do all data transformation. Nothing changes the data without passing through a logged tool.
- **Reports come from logs, not from the LLM.** Every number in a report is injected from tool output. LLM text is limited to interpretation and is visibly separated from computed results.
- **Raw data is immutable.** Loaded once, hashed, stored read-only. Everything else is derived and can be regenerated.
- **Every change is accounted for, and anything not accounted for did not change.** Enforced with row accounting, cell diffs, untouched-column hashes and replay.
- **Deterministic checks first, judgment second.** Each gate runs contract checks, then invariant checks, and only then LLM or human judgment.
- **Flag rather than fix.** Clear errors are fixed; judgment calls (outliers, heavy imputation, dropping columns) are flagged and decided later with the intent in mind.
- **The LLM never sees raw rows.** It sees profiles, schemas and anonymized or synthetic sample values. PII is detected and masked before anything leaves the machine.
- **Measure everything.** The system is judged by benchmark numbers, not by demos.

2. Architecture overview

The pipeline is a linear chain with one bounded loop-back (review → cleaning, max 2 rounds) and one human checkpoint. Every stage writes to a shared audit trail, and the final report is assembled from that trail. Publishing is a separate, human-curated step after the run.



Stage	Type	Core responsibility	Main output
0 Input	Human	Describe data, goal, focus, exclusions	Context brief (YAML/form)
1 Ingestion & profiling	Deterministic	Load, hash, profile, verify brief claims, detect PII	Raw layer + profile JSON
2 Cleaning agent	LLM + tools	Fix clear errors, flag judgment calls	Clean data + data report + handoff JSON
3 Review & planning agent	LLM + tools	Validate cleaning, loop back if needed, plan analysis	Analysis plan JSON
4 Human approval	Human	Accept or edit the plan	Approved plan
5 Analysis agent	LLM + tools	Execute plan: EDA, baselines, models, explainability	Analysis report + model artifacts
6 Final report	Deterministic	Assemble reports and audit trail; save artifact catalog	HTML report bundle + artifact catalog
7 Publish	Human + deterministic	Select artifacts, run publish gate, build and deploy dashboard	Versioned dashboard on GitHub Pages

3. Pipeline stages in detail

Each stage lists its purpose, inputs, what it does, outputs, and what is needed to build it.

Stage 0 — Context brief (human input)

Purpose. Give the system the domain knowledge it cannot infer from the data: what the rows mean, units, known issues, the goal, and what is off-limits.

Inputs	Free-text description plus optional structured fields.
What it does	Fields: data description; goal/question; target if known; column notes or uploaded data dictionary; known issues; focus areas; excluded variables (e.g. legally sensitive features); unit of observation and ID column. Every checkable claim is treated as a hypothesis and verified in Stage 1 (e.g. 'cust_id is unique', 'covers 2019–2024', 'target is binary'). If the brief is thin, the planner states its interpretation and the human corrects it at approval.
Outputs	Context brief object, stored in the run manifest and passed to every agent.
Needed to build	Pydantic schema for the brief; YAML loader (from Phase 0); Streamlit form (Phase 7).

Stage 1 — Ingestion & profiling (deterministic)

Purpose. Load any supported tabular file reliably, preserve the raw data, and produce the profile every later stage works from.

Inputs	Data files, context brief.
What it does	Format detection and loading into DuckDB; raw layer stored read-only with file hash and data hash; stable row IDs assigned; schema and type inference with coverage per column; profiling (nulls, cardinality, distributions, outliers, correlations); verification of brief claims; PII detection on column names and values (emails, phone numbers, CPR numbers) with masking before anything reaches the LLM; locale checks for decimal separators and date formats.
Outputs	Raw layer, profile JSON (compact summaries for the LLM, full detail in the log), claim verification results, PII report.
Needed to build	Loaders (CSV, Excel, Parquet, JSON); profiler; PII detector (e.g. Presidio plus custom regex for CPR); unit tests and golden files.
Quality gate	Gate 1: rows loaded = rows in source; parse errors below 0.1% (block above 1%); unique column names; types covering at least 95% of values or flagged; provenance hashes present.

Stage 2 — Cleaning agent

Purpose. Turn raw data into analysis-ready data through explicit, logged operations, without making silent or unjustified changes.

Inputs	Profile JSON, context brief, claim verification results, flags from Stage 1.
What it does	The agent reads the profile and outputs an operation plan chosen from a fixed op library (~15 ops: cast, parse_dates, normalize_decimal, drop_duplicates, standardize_categories, impute, clip_outliers, drop_column, rename, etc.). Tools execute each op and log parameters, code and before/after statistics. Clear errors are fixed; judgment calls are flagged. A sandboxed custom-code escape hatch exists for the long tail and is highlighted in the report. Silent coercion is banned. Canary rows with known correct outputs are injected, checked and removed on every run.
Outputs	Clean dataset, human-readable data report, machine-readable handoff JSON (schema, types, null rates, applied transformations, flags), dropped_rows table.
Needed to build	Op library with unit tests; LangGraph node; prompt with profile-only input; Pandera schemas; sandbox (separate container, no network, resource limits); Jinja2 report template.
Quality gate	Gate 2: schema contract; row loss under 5% pass, 5–20% warn, over 20% block; target never imputed; no unexplained new nulls; distribution shift (PSI > 0.2 or KS $p < 0.01$) flagged; primary key intact; replay of logged ops reproduces the output hash.

Stage 3 — Review & planning agent

Purpose. Independently check the cleaning result, then decide what is worth finding out, given the clean data and the user's intent.

Inputs	Clean data profile, cleaning log, flags, handoff JSON, context brief.
What it does	Review: deterministic checks run first (reconciliation, leakage heuristics such as near-perfect feature–target correlation, target protection). The agent reads results and flags, then accepts or sends specific fixes back to the cleaning agent (max 2 rounds). Planning: writes a structured plan with restated objective and assumptions; target and task type (or a statement that prediction is not supported); 3–5 prioritized questions tied to specific columns; excluded features with reasons; methods chosen only from Agent 3's tool registry; success criteria; data limitations.
Outputs	Review verdict and fix requests (if any), analysis plan JSON.
Needed to build	Plan schema (Pydantic); tool registry the planner can reference; loop-back edge in LangGraph; leakage heuristics.
Quality gate	Gate 3: valid plan schema; every referenced column exists; every method exists in the registry; target not a feature; leaky columns excluded; 3–5 questions; feasibility (e.g. at least 10 minority-class events per feature, at least 30 rows per segment) or item marked infeasible.

Stage 4 — Human approval

Purpose. Let the user correct the system at the moment it matters most: after the data is understood, before effort is spent on analysis.

Inputs	Analysis plan, data report summary, claim verification results.
What it does	User accepts, edits or rejects plan items. Edits are recorded in the audit trail. Optional per-step review mode for users who want to approve cleaning as well.
Outputs	Approved plan.
Needed to build	LangGraph interrupt; simple review UI (CLI first, Streamlit later).

Stage 5 — Analysis agent

Purpose. Execute the approved plan and produce trustworthy exploratory and predictive results.

Inputs	Clean data, approved plan, context brief, handoff JSON.
What it does	Intent-driven EDA per plan question; dummy baseline; interpretable model (logistic/linear); gradient boosting (LightGBM) or AutoML (FLAML); cross-validation with fold variance; SHAP explanations; segment analysis for focus areas. Items that turn out infeasible are marked with a reason instead of forced. LLM interpretation is kept separate from computed numbers.
Outputs	Analysis report, figures, metrics, model artifacts, per-item status.
Needed to build	EDA and modeling tools in the registry; scikit-learn, LightGBM/FLAML, SHAP; report template; number-traceability checker.
Quality gate	Gate 4: every plan item done or infeasible; model compared with baseline ('no signal' is a valid result); train–test gap above 0.1 flagged; suspiciously high performance (e.g. AUC above 0.95) triggers leakage investigation; fold variance reported; every number in the narrative matches a logged value.

Stage 6 — Final report

Purpose. Deliver the analysis together with the evidence behind it.

Inputs	All reports, audit trail, run manifest.
What it does	Assembles: context brief and verification results; data report; review verdict; approved plan; analysis report; gate results; data card and model card generated from logs; appendix with full code and parameters per step.
Outputs	HTML report (PDF export optional), the full audit bundle, and the artifact catalog: every chart, table, metric and finding as a standalone unit with its own aggregated data and audit references.
Needed to build	Jinja2 templates; data card and model card templates; artifact contract.

Stage 7 — Publish to GitHub Pages (human-curated)

Purpose. Let the user choose which results go live, and guarantee that nothing sensitive or untraceable is published.

Inputs	Artifact catalog from a run; a publish config written by the user.
What it does	The user selects artifacts from the catalog (in the Streamlit publish builder, or directly in a YAML config under publish/), orders them, and optionally adds commentary, which is labeled as human-written. The publish gate runs deterministic checks. A builder renders the selection into a self-contained static dashboard with interactive Plotly charts, a provenance panel per artifact, and the gate results. Each publish is an immutable snapshot (site/dashboards/<slug>/v<N>/); the slug root shows the latest version. A project page (site/index.html) lists all dashboards with headline results. Committing site/ triggers a GitHub Actions workflow that deploys to GitHub Pages.
Outputs	Versioned dashboard at https://<username>.github.io/<repo>/dashboards/<slug>/, updated project page, committed publish config as the record of what went live.
Needed to build	Artifact and publish-config contracts; publish gate; Jinja2 dashboard and project-page templates; CLI (python -m pipeline.publish); GitHub Actions Pages workflow; Streamlit selection page (Phase 7).
Quality gate	Publish gate: every selected artifact exists in the run's catalog; no row-level data; nothing marked restricted; no PII in values, column names or text; every group has n of at least 10; every artifact has audit references; open flags explicitly acknowledged in the config. Any failure blocks publishing.

4. Ingestion & profiling: detailed design

Stage 1 is the foundation everything else trusts. It is fully deterministic: no LLM. Its only job is to get the data in without changing it, describe it accurately, and hand the cleaning agent a compact, honest picture. Think of it as receiving a package: unpack it, photograph everything exactly as it arrived, count it, and label each item with what it seems to be, but change nothing.

4.1 Sub-steps

Step	What it does	Output
1 Detect	Format from extension; encoding (UTF-8, UTF-8 with BOM, cp1252 for older Danish files with æøå); CSV delimiter (, ; tab |) by consistency of field counts over the first 50 lines; Excel sheets	FileInfo
2 Load raw	CSV and Excel values stored as literal text in DuckDB schema raw. Add _row_id and _source_line. Unreadable lines go to raw.<table>__rejects with a reason. Compute file hash and data hash	raw.<table>, LoadResult
3 Structural checks	Rows loaded + rejected = rows in source; reject rate; duplicate or blank headers renamed deterministically and reported; empty columns	Gate 1 results
4 Propose types	Try candidate parsers per column and measure coverage; propose the most specific type parsing at least 95% of non-null values; otherwise flag ambiguous with failing examples. Nothing is converted	TypeProposal per column
5 Profile	Per column: missing values incl. null tokens, distinct count, top values, min/quantiles/max, far outliers (3×IQR), sentinel candidates. Per dataset: rows, duplicates, candidate keys	DatasetProfile
6 Verify brief	Each checkable claim in the context brief becomes a test (target exists, ID unique, period matches, excluded columns exist)	claim_mismatch flags
7 Detect PII	Column-name hints; value patterns: email, CPR (first six digits must be a valid date), Danish phone numbers, IBAN; Presidio for names in free text later. Flag and mask, never drop	PII level per column
8 Compact profile	One or two lines per column for the LLM (~100–150 tokens), personal values masked to their shape	profile_llm.json

4.2 Why load everything as text

Most tools guess types while reading, and that is where silent corruption happens: '007' becomes 7 and the ID is destroyed; the Danish amount '1.234,50' (= 1234.50) is misread; unparseable dates become empty. Storing the literal text means nothing is lost. Types are applied later by the cleaning agent as logged cast operations, which the reconciliation checks can verify. Parquet files are the exception: they carry their own types.

4.3 Type inference

Candidate	Details
Integer / float	Decimal comma vs dot decided per column from unambiguous values (e.g. '12,5' or '1,234.5'); thousands separators; currency symbols stripped
Boolean	true/false, yes/no, ja/nej, y/n, 1/0 with at most two distinct values
Date	ISO plus common formats; mixed formats reported ('4/5 match %d-%m-%Y'); if no day above 12 is seen, day/month order is flagged as ambiguous
ID	Unique values and leading zeros, an ID-like name (id, nr, key, nummer), or code-like text; kept as text

Candidate	Details
Categorical	Low cardinality relative to rows
Text	Everything else
Null tokens	'', NA, N/A, null, none, -, ?, missing: counted as missing, not converted
Sentinels	999, 9999, -1, -999, 1900-01-01, 9999-12-31: flagged; only the brief or the cleaning agent decides whether they mean missing

4.4 Example

A Danish CSV export with columns kunde_id;alder;beløb;dato;email should produce proposals like these:

Column	Proposal	Notes
kunde_id	id	Unique, leading zeros, keep as text
alder	integer	20% missing; null token 'NA'
beløb	float	Decimal comma; possible sentinel 999
dato	date	Mixed formats: 4/5 match %d-%m-%Y; flagged
email	categorical	80% missing; PII likely; values masked for the LLM

4.5 Fixture set: build this first

Create small, deliberately nasty files, each with a known expected result, and test every loader and profiler function against them. When real data breaks something, first add a fixture that reproduces it, then fix the code.

- Semicolon-delimited with decimal commas, leading-zero IDs, NA tokens, a 999 sentinel, an email column
- UTF-8 with a byte-order mark (typical Excel export)
- cp1252/latin-1 file containing æøå
- Quoted field containing a newline
- Duplicate and blank column headers
- A line with too many fields (must be rejected, not lost)
- Dates where no day exceeds 12 (day/month ambiguous)
- Valid-looking CPR numbers
- Excel file with two sheets and text IDs like '001'
- A typed Parquet file

4.6 Design decisions

Decision	Choice	Reason
Types at load	No; literal text first	Nothing can be coerced silently
Profiler	Own, structured profiler	Output must be compact for the LLM and structured for the log; ydata-profiling is built for humans and slow (optional human report later)
Full data vs sample	Counts on full data; expensive stats on a 100k-row sample	Accurate where it matters, fast on large files; sampling recorded in the manifest

Decision	Choice	Reason
CSV parser	Python csv module initially	Exact control of rejects, line numbers and encodings; switch to DuckDB's native reader (all_varchar, store_rejects) for very large files
PII	Flag and mask, never auto-drop	Dropping is a judgment call for later, logged stages

Build order: detection and loaders → raw layer with hashes and lineage → Gate 1 → type inference → profiling → claim verification → PII → compact profile.

5. Quality assurance

5.1 Gate structure

Every gate runs three layers in order, and uses one consistent failure policy.

Layer	Question	How
Contract	Is the output well-formed?	Pydantic and Pandera schemas
Invariant	Did something break or get lost?	Deterministic rules with thresholds
Judgment	Is it sensible?	LLM review or human, only after the first two pass

Outcome	Meaning
Block	Stop the run and report why
Retry	Send the error back to the agent, max 2 attempts
Flag	Continue, but show the issue prominently in the report

5.2 Protecting the data from the first stages

You cannot prove the cleaned data is correct, but you can guarantee that every change is accounted for. What remains are wrong-but-documented decisions, which the review agent, invariants and benchmark are designed to catch.

Safeguard	Rule
Immutable raw layer	Raw data is read-only and hashed; every run can restart from it
Row accounting	rows_raw = rows_clean + rows_dropped exactly; every dropped row has an op and a reason
Cell diff	Changed cells per column must equal what the logged ops claim
Untouched columns	Columns no op touched must hash identically to raw
Aggregate reconciliation	Changes in sums, means and quantiles must be explained by the ops
Invariants	Ranges, categories, uniqueness and cross-column rules derived before cleaning must still hold
Replay	Replaying the op log on raw data reproduces the exact output hash
Canary rows	Synthetic rows with known correct outputs checked on every run
Sensitivity check	Compare key stats and a baseline model on minimally cleaned vs agent-cleaned data; large differences must be explained by specific ops

5.3 Classic silent failures to guard against

Risk	Example	Check
Decimal separators	Danish 1.234,5 read as 1.2345	Magnitude check against raw strings; parse success rate
Date formats	03/04 read as 4 March instead of 3 April	Share of day values above 12; flag if ambiguous
ID casting	'007' becomes 7	Round-trip check for ID columns
Silent coercion	Unparseable values become NaN, then get imputed	New nulls after a cast must be zero or logged per value
Joins	Rows duplicated by non-unique keys	Row count before and after every join

6. Cross-cutting requirements

Requirement	What	Phase
Run manifest	Git SHA, prompt versions, model name and version, seeds, config, context brief per run. Makes decisions reproducible, not just ops	0
Tracing	Langfuse (self-hostable) or LangSmith for every LLM call, tool call and token count	0
PII handling	Detect and mask personal data before LLM calls; CPR-aware	1
Prompt injection	Data values are delimited and treated as data; suspicious strings stripped; data text never triggers tool calls	2
Sandbox	Custom code runs in an isolated container with no network and resource limits	2
Cost and limits	Token and cost tracking (target under $0.50 per dataset); size limits and sampling for profiling above ~1 GB; per-agent timeouts	2–5
Testing and CI	GitHub Actions: op unit tests, reconciliation tests, end-to-end smoke test with a mocked LLM	1 onward
Documentation	Data card and model card generated from logs, in the style of model risk documentation	5–6
Publishing	Only public datasets; aggregates only; publish gate; immutable versioned snapshots on GitHub Pages	0, 7

7. Tech stack

Layer	Tools
Language & data	Python, DuckDB, pandas or Polars
Orchestration	LangGraph (LangChain ecosystem)
Validation	Pydantic (contracts), Pandera (data schemas)
Modeling	scikit-learn, LightGBM, FLAML (optional), SHAP
Privacy	Presidio + custom regex
Observability	Langfuse or LangSmith
Reporting	Jinja2 → HTML (PDF export optional), matplotlib/plotly
Interface & packaging	FastAPI, Streamlit, Docker Compose, GitHub Actions
Publishing	Plotly (charts), Jinja2 (static dashboards), GitHub Pages

8. Suggested repository structure

```
pipeline/
  contracts/      context_brief.py, handoff.py, plan.py, audit.py, manifest.py, artifact.py
  ingestion/      loaders.py, profiler.py, pii.py, claims.py
  ops/            op library, one module per op family, each logged
  checks/         gates.py, reconciliation.py, leakage.py, traceability.py
  agents/         cleaning.py, review_planning.py, analysis.py, prompts/
  tools/          eda.py, modeling.py, explain.py, registry.py
  graph.py        LangGraph wiring, loop-back, interrupt
  reports/        templates/ (Jinja2), builder.py
  publish/        gate.py, builder.py, site.py, templates/, CLI
publish/          committed publish configs (what went live)
site/             generated static site, project.yaml
.github/workflows ci.yml, pages.yml
benchmark/        injectors/, datasets/, planted_signals/, run_benchmark.py, metrics.py
app/              api.py (FastAPI), ui.py (Streamlit)
tests/            unit/, integration/, e2e_mocked/
docker/           Dockerfile, docker-compose.yml, sandbox/
```

9. Roadmap

Estimates assume 15–20 hours per week; full-time roughly halves them. Total about 12 weeks including the additions above; keep 20–30% slack inside each phase for debugging agent behavior.

Phase	Weeks	Deliverable	Done when
0 Contracts & setup	1	Repo, Docker, schemas incl. artifacts, manifest, tracing, datasets, Pages workflow	All contracts exist with example instances; demo dashboard is live on GitHub Pages
1 Deterministic foundation	2	Ingestion, profiling, claims, PII, op library, reconciliation, Gate 1	A dataset can be cleaned by hand with ops; replay and row accounting pass
2 Cleaning agent	1.5	LLM op selection, Gate 2, sandbox, data report	Runs end to end on 5 datasets without unexplained blocks
3 Corruption benchmark	1	Injectors and metrics	First detection and repair numbers
4 Review & planning agent	1	Review, loop-back, plan, approval, Gate 3	Plans pass Gate 3 on all benchmark datasets
5 Analysis agent	1.5	Plan execution, models, Gate 4, analysis report, artifact catalog	Gate 4 passes; every output is a catalogued artifact
6 Evaluation & ablations	1.5	Full results incl. comparison baselines	All metrics and ablations reported
7 Interface, publishing & write-up	2.5	Streamlit app with publish builder, published dashboards, project page, write-up	2–3 dashboards live; a stranger can run it from Docker in under 10 minutes

Phase 0 — Contracts & setup (week 1)

Goal. Fix the interfaces before writing logic, so every later component has a clear target.

- Repository structure, Docker, LangGraph skeleton with empty nodes
- Pydantic models: context brief, handoff (clean data + profile + flags), analysis plan, audit-log record, run manifest
- Tracing set up (Langfuse or LangSmith)
- Artifact and publish-config contracts; GitHub Pages workflow; demo dashboard deployed to verify the path end to end
- Dataset selection: ~5 clean public tabular sets with a credit/risk lean (e.g. German Credit, Taiwan credit card default, a Lending Club sample) plus 2–3 genuinely messy real datasets

Needed: Python environment, Docker, LLM API key, tracing account or self-hosted instance.

Exit criteria: Each contract has a schema and one realistic example instance; the empty graph runs.

Phase 1 — Deterministic foundation (weeks 2–3)

Goal. Build everything that does not need an LLM, and make it trustworthy.

- Loaders for CSV, Excel, Parquet, JSON into DuckDB; immutable raw layer with hashes and row IDs
- Profiler with compact LLM summaries and full logged detail
- Claim verification against the context brief; PII detection and masking; locale checks
- Op library (~15 ops), each logging parameters, code and before/after stats; silent coercion banned
- Reconciliation: row accounting, cell diff, untouched-column hashes, replay; Gate 1
- Unit tests for every op; CI pipeline

Needed: DuckDB, pandas/Polars, Pandera, Presidio, pytest, GitHub Actions.

Exit criteria: You can clean a dataset manually through ops and get a complete audit log; replay and row accounting pass on all datasets.

Phase 2 — Cleaning agent (weeks 4–5a)

Goal. Let the LLM choose cleaning operations while the system guarantees nothing silent happens.

- LangGraph node with profile-only input and structured op-plan output
- Execution through tools, Gate 2, retry loop with error feedback (max 2)
- Flagging of judgment calls; canary rows; sandboxed escape hatch
- Prompt-injection defenses on data values; cost tracking
- Jinja2 data report and handoff JSON

Needed: LLM API, op library from Phase 1, sandbox container, report templates.

Exit criteria: Runs end to end on the 5 clean datasets with no unexplained blocks; reports are complete.

Phase 3 — Corruption benchmark (weeks 5b–6a)

Goal. Measure the cleaning agent before building on top of it.

- Injectors: missing values, duplicates, type errors, decimal-comma and date-format issues, outliers, a leaky column
- Metrics: detection rate, correct repair rate, false alarm rate, harmful ops
- Hand-labeled issues for the messy real datasets
- Tune prompts and ops based on results

Needed: Clean datasets from Phase 0, injector framework, labeled messy data.

Exit criteria: Baseline numbers exist for synthetic and real messy data, and the main failure modes are known.

Phase 4 — Review & planning agent (weeks 6b–7a)

Goal. Add independent validation and turn data understanding plus intent into an executable plan.

- Deterministic review checks and leakage heuristics
- Agent review verdict and loop-back to cleaning (max 2 rounds)
- Plan generation constrained to the tool registry; Gate 3
- Human approval via LangGraph interrupt (CLI first)

Needed: Plan schema, tool registry definition, loop-back edge.

Exit criteria: Plans pass Gate 3 on all benchmark datasets; loop-back fixes at least some injected issues the cleaning agent missed.

Phase 5 — Analysis agent (weeks 7b–8)

Goal. Execute approved plans and produce trustworthy results.

- EDA tools per question type; dummy and interpretable baselines; LightGBM or FLAML
- Cross-validation with fold variance; SHAP; segment analysis
- Infeasible-item handling; Gate 4 including overfitting, leakage alarm and number traceability
- Every chart, table, metric and finding emitted as an artifact with aggregated data and audit references
- Analysis report; data card and model card

Needed: scikit-learn, LightGBM/FLAML, SHAP, report templates.

Exit criteria: Gate 4 passes on all benchmark datasets; every number in the narrative traces to a logged value.

Phase 6 — Evaluation & ablations (weeks 9–10a)

Goal. Produce the evidence that makes the project credible.

- Full corruption benchmark and messy real datasets
- Planted-signal datasets: real effect, spurious correlation, interaction; check what the analysis finds and overclaims
- Stability: 5 repeated runs per input; measure decision consistency
- Context-brief experiment: good brief vs none vs deliberately wrong
- Comparison baselines: single-agent code-interpreter setup and plain AutoML on the same benchmark
- Ablations: review agent on/off; profile-only vs profile + anonymized samples

Needed: Benchmark from Phase 3, planted-signal generator, comparison setups.

Exit criteria: All metrics and ablations are in tables ready for the write-up.

Phase 7 — Interface, publishing & write-up (weeks 10b–12)

Goal. Make it usable, publish the best results, and present the project.

- FastAPI backend; Streamlit UI: upload, context brief form, plan approval, report and audit-trail browser
- Streamlit publish builder: browse a run's artifacts, select and order them, add labeled commentary, preview, run the publish gate, write the publish config
- Publish 2–3 showcase dashboards from public datasets; fill in the project page with headline benchmark results
- Docker Compose; README with quickstart; cached demo runs
- Write-up organized around measurements and tradeoffs; short demo video; link the project page from CV and LinkedIn

Needed: Everything above; GitHub Pages enabled on the repository.

Exit criteria: 2–3 dashboards are live; a stranger can run the system from Docker in under 10 minutes and understand the results from the project page.

10. Evaluation plan

What	How	Metrics
Cleaning quality	Injected corruption on clean datasets; hand-labeled messy real data	Detection rate, correct repair rate, false alarms, harmful ops
Review value	Review agent on vs off	Additional issues caught, harmful decisions prevented
Plan quality	Gate 3 pass rate; human edit rate at approval	Pass rate, edits per plan, infeasible items
Analysis quality	Planted-signal datasets	Real effects found, spurious effects overclaimed
Model quality	Against dummy baseline and hand-tuned reference	CV metric and gap to reference
Hallucination	Number traceability	Share of narrative numbers matching logs
Stability	5 runs per input	Agreement of op choices, plans and conclusions
Context value	Good vs missing vs wrong brief	Outcome change; wrong claims detected
Comparison	Single-agent code interpreter; plain AutoML	All of the above on the same benchmark
Cost	Per run	Tokens, dollars, wall-clock time

11. Scope, risks and cut line

Out of scope for v1

- Non-tabular data (text, images, audio)
- Time-series-specific modeling and forecasting
- Multi-table joins beyond simple keys
- A separate reviewer-of-the-cleaner agent, unless the benchmark shows it is needed
- Live public runs with uploads; scheduled dashboard refreshes (candidate for v2)

Main risks

Risk	Mitigation
Scope creep	Tabular only; one prediction task per run; fixed op library
Vague handoff contracts	Define contracts in Phase 0 before any logic
Cleaning agent context overload on wide data	Compact tool summaries; test at 50+ columns
Overambitious plans	Cap at 3–5 questions; methods only from the registry
Rubber-stamp review	Measure extra issues caught; drop review if near zero, keep planning
Overclaiming in reports	Numbers injected from logs; interpretation labeled; traceability gate
Real messy data harder than synthetic	Hand-labeled real test set; report results honestly
Timeline slip	Built-in slack; cut line below

Cut line. The end of Phase 3 is already a showable project: an audited, benchmarked cleaning agent with full reconciliation. Everything after it adds depth. If time runs short, finish Phase 3 well rather than Phase 7 badly.

12. Building it with Claude Code

Build the project on your own machine with Claude Code, using this document as the reference. Put it in the repository as docs/project_spec.pdf and create a CLAUDE.md at the repository root, which Claude Code reads at the start of every session.

12.1 Suggested CLAUDE.md

```
# Project: auditable agentic data science pipeline
Reference: docs/project_spec.pdf. Read the relevant section before larger changes.

## Principles (do not violate)
- The LLM decides, tools execute: no data transformation outside logged ops in pipeline/ops/.
- Every op returns (new_table, OpRecord) with params, exact code, before/after stats.
- Raw data is read-only. Never write to the raw schema or data/raw/.
- CSV/Excel are loaded as literal text; types are applied later as logged cast ops.
- No silent coercion (never errors="coerce"); new nulls must be explained.
- Numbers in reports come from logs, never from LLM text.
- The LLM never sees raw rows or personal data; only compact, masked profiles.
- Deterministic checks first, judgment second. Flag judgment calls; don't fix them.

## Working rules
- One roadmap task per session. Propose a plan before writing code.
- Tests first for ops, checks and gates; use the fixture set in tests/fixtures/.
- Run pytest before every commit. Small, focused commits.
- Contracts in pipeline/contracts/ are the source of truth; change them deliberately.
- Record design choices in DECISIONS.md (choice, alternatives, reason).
```

12.2 How to work

- **One small task per session**, taken from the roadmap, e.g. 'implement CSV loading into the raw schema with row IDs, source lines, rejects and hashes'. Vague large tasks produce vague large code.
- **Plan first.** Ask for an approach before code and push back on it.
- **Tests first.** Write or review the test cases yourself; they define what correct means.
- **Read every diff** before committing, not just skim it.
- **Divide the work deliberately.** Let Claude Code do boilerplate: loaders, report templates, Streamlit UI, Docker, CI. Write the core yourself or co-write it line by line: the op contract, reconciliation checks, the gates, and the benchmark metrics.
- **Explain it back.** After each phase, explain the code without looking at it. If you can't, you don't own it yet, and an interviewer will notice.
- **Keep DECISIONS.md.** It becomes most of the write-up.

12.3 First sessions

Session	Task
1	Repository setup: pyproject, virtual environment, folder structure (section 8), pytest, GitHub Actions CI, CLAUDE.md, DECISIONS.md
2	Contracts as Pydantic models: context brief (with YAML loader), audit log (op record, gate result), handoff, analysis plan, run manifest, artifact and publish config, each with tests
3	Fixture set from section 4.5 with expected results
4	Detection and loading into the raw schema; Gate 1 structural checks
5	Type inference with coverage and failing examples

Session	Task
6	Profiler, claim verification, PII detection, compact LLM profile, CLI that writes JSON outputs
7+	Op library, then reconciliation checks (row accounting, cell diff, untouched-column hashes, replay); then Phase 2

13. Final reflections

- **The idea is good; the evidence makes it strong.** 'An AI that cleans and analyzes data' is a crowded idea. The audit trail and the benchmark numbers are what set this project apart. If something must be cut, cut features, never the evaluation.
- **The boring parts matter most.** Ingestion, the op library and reconciliation feel slow next to building agents, but they are what make the system trustworthy, and they are what most similar projects skip. Don't rush Phase 1.
- **Expect real data to humble the system.** The cleaning agent will look much better on synthetic errors than on genuinely messy data. Report both honestly; an explained gap is more convincing than a perfect number.
- **Scope creep is the most likely failure.** Twelve part-time weeks is realistic only with tabular data, a fixed op library, and no features the benchmark hasn't shown a need for.
- **Failures are material.** Keep notes on every mistake the agents make. They are the best interview stories and the best write-up content.
- **It fits a coherent profile.** Risk-model thesis work, a RAG project, and an auditable analysis pipeline for regulated data make one story for banks, insurers and fintechs. Lead with that framing on the project page and CV.

One-line summary. Build the deterministic foundation carefully, keep every change accounted for, measure everything, and let the audit trail be the product.